

HỆ THỐNG NHÚNG (ESYS431080)

TẬP LỆNH CỦA ARM

ThS. ĐINH CÔNG ĐOAN



- **Tóm tắt:** Chương này trình bày tập lệnh của ARM
- **Mục tiêu:** sau khi học xong chương này, người học có khả năng:
 - Trình bày được các đặc điểm của tập lệnh ARM
 - Trình bày được quá trình thi hành lệnh
 - Trình bày được các cách tổ chức trật tự byte trong ARM
 - Trình bày được định dạng tổng quát của 1 lệnh
 - Trình bày được cách làm việc của 1 số lệnh cơ bản

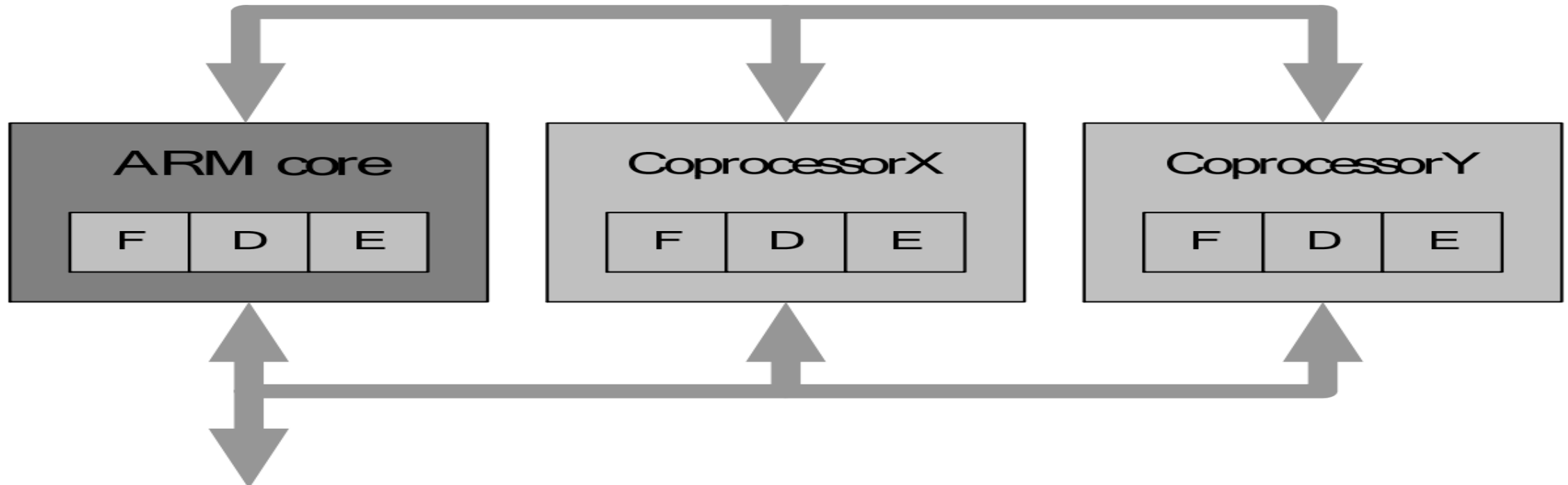
- **Nội dung:**

- 1. Các đặc điểm của tập lệnh ARM**
- 2. Ôn tập về các chế độ làm việc, các thanh ghi của ARM**
- 3. Quá trình thi hành lệnh**
- 4. Trật tự byte**
- 5. Định dạng tổng quát của 1 lệnh**
- 6. Thực thi có điều kiện**
- 7. Một số lệnh cơ bản: các lệnh di chuyển (MOV, LDR), các lệnh số học (ADD, ADC, SUB, SBB...) các lệnh logic (AND, ORR, EOR...) các lệnh nhân chia (MUL, UMULL, MLA, DIV...) dịch và quay (LSL, ROR, RRX...), lệnh rẽ nhánh và gọi chương trình con (B, BX, CALL...) lệnh lặp vòng (LOOP) lệnh so sánh (CMP...) các lệnh trên stack (PUSH, POP)**
- 8. Các chế độ định địa chỉ**

1. Tất cả các lệnh có chiều dài 32 bit
2. Hầu hết các lệnh thực thi trong 1 chu kỳ
3. Hầu hết các lệnh có thể thực thi có điều kiện
4. Kiến trúc Load/Store
5. Các lệnh xử lý dữ liệu chỉ thực hiện trên các thanh ghi
 - Định dạng 3 toán hạng
 - Kết hợp ALU và bộ dịch (Shifter) để xử lý bit tốc độ cao
6. Các lệnh truy cập bộ nhớ cụ thể với các chế độ định địa chỉ mạnh mẽ (tự động chỉ mục)
 - Các loại dữ liệu 32 bit và 8 bit
 - Kiểu dữ liệu 16 bit trên ARM v4
 - Các lệnh load và store nhiều thanh ghi
7. Tập lệnh mở rộng thông qua các bộ đồng xử lý
8. Tập lệnh nén thumb 16 mật độ cao



1. Có thể lên tới 16 bộ đồng xử lý có thể được định nghĩa
 - Mở rộng tập lệnh ARM
 - Mỗi bộ đồng xử lý có thể có 16 thanh ghi riêng
 - Có kiến trúc Load/Store



1. Thumb là tập lệnh 16 bit
 - Được tối ưu cho mật độ mã, cụ thể là code C
 - Hiệu suất cải tiến từ bộ nhớ hạn chế
 - Tập con chức năng của tập lệnh ARM
2. Lỗi có 2 trạng thái thực thi: ARM và Thumb
 - Có thể chuyển giữa chúng bằng cách dùng lệnh BX
3. Thumb có các đặc trưng
 - Hầu hết các lệnh thumb được thực thi không điều kiện
 - Nhiều lệnh thumb xử lý dữ liệu sử dụng định dạng 2 địa chỉ
 - Các Định dạng tập lệnh thumb thì ít quy tắc hơn các định dạng tập lệnh ARM, dẫn đến mật độ mã hoá dày



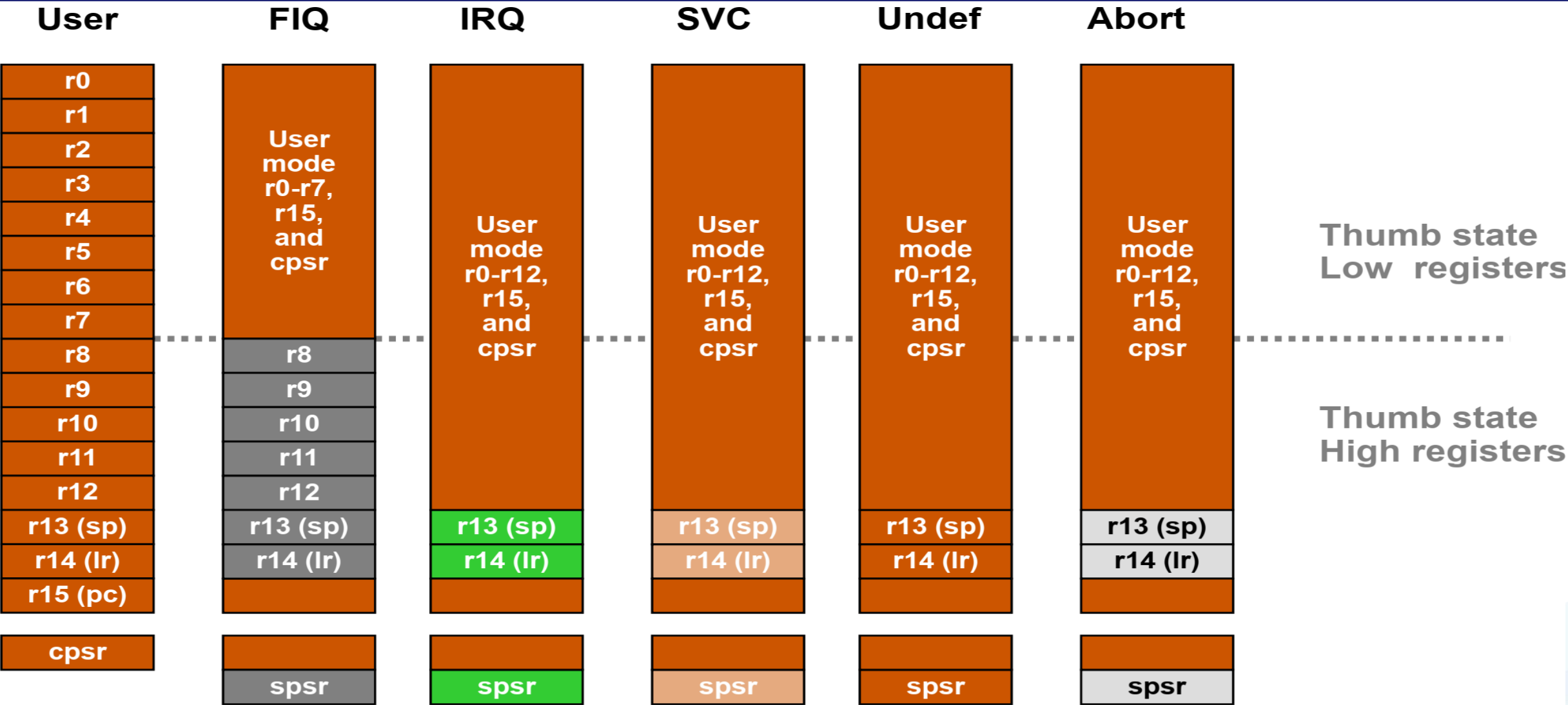
1. ARM có 6 chế độ làm việc

- User – chế độ không đặc quyền, mà trong đó hầu hết các tác vụ được chạy
- FIQ – vào chế độ này khi xảy ra ngắt có độ ưu tiên cao
- IRQ – vào chế độ này khi xảy ra ngắt có độ ưu tiên thấp
- Supervisor – vào chế độ này khi reset và khi 1 lệnh ngắt mềm xảy ra
- Abort – được dùng để xử lý các vi phạm truy cập bộ nhớ
- Undef – được dùng để xử lý các lệnh chưa được định nghĩa

2. Kiến trúc ARM version 4 thêm chế độ làm việc thứ bảy

- System – chế độ đặc quyền sử dụng chung các thanh ghi như chế độ User





Note: System mode uses the User mode register set



Abort Mode

r0
r1
r2
r3
r4
r5
r6
r7
r8
r9
r10
r11
r12
r13 (sp)
r14 (lr)
r15 (pc)
cpsr
spsr

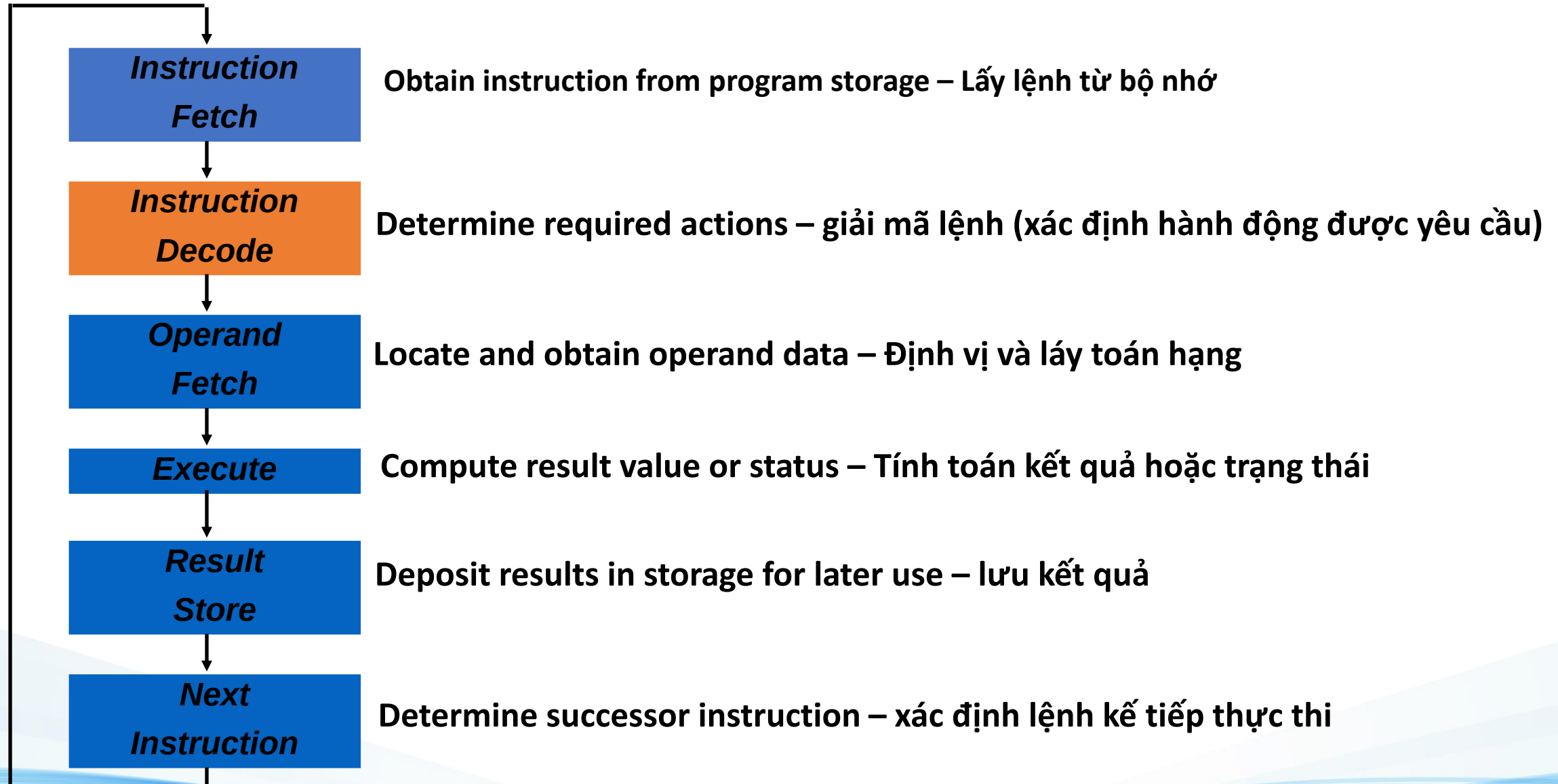
Banked out Registers

User	FIQ	IRQ	SVC	Undef
	r8			
	r9			
	r10			
	r11			
	r12			
r13 (sp)	r13 (sp)	r13 (sp)	r13 (sp)	r13 (sp)
r14 (lr)	r14 (lr)	r14 (lr)	r14 (lr)	r14 (lr)
	spsr	spsr	spsr	spsr



1. Tất cả các lệnh có thể truy cập trực tiếp các thanh ghi $R_0 - R_{14}$
2. Hầu hết các lệnh cũng cho phép sử dụng thanh ghi PC
3. Một số lệnh cụ thể cũng cho phép truy cập tới các thanh ghi CPSR và SPSR
4. Lưu ý: khi ở chế độ đặc quyền, nó cũng có thể Load – Store các thanh ghi của chế độ người dung tới hoặc từ bộ nhớ



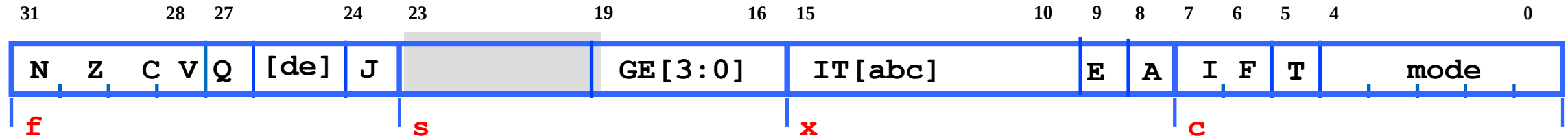


1. Bộ nhớ được xem như một mảng tuyến tính các byte, được đánh địa chỉ từ $0 - 2^{32} - 1$
2. Kích thước dữ liệu:
 - Word – 4 byte
 - Half – word – 2 byte
 - Byte – 8 bit
3. Trật tự byte: có thể là Little Endian hoặc Big Endian

0x00000000	00
0x00000001	10
0x00000002	20
0x00000003	30
0x00000004	FF
0x00000005	FF
0x00000006	FF
	
0xFFFFFFF	00
0xFFFFFFF	00
0xFFFFFFF	00

1. Trật tự byte: cách thức lưu dữ liệu kích thước lớn hơn 1 byte vào bộ nhớ
2. Có 2 cách: Little Endian và Big Endian
3. Big Endian: byte có trọng số thấp được lưu vào ô có địa chỉ cao, byte cao được lưu vào địa chỉ thấp
 - Ví dụ: số có kích thước 4 byte bắt đầu tại địa chỉ 0x00000000 là 00102030
4. Little Endian: Byte thấp được lưu vào địa chỉ thấp, byte cao được lưu vào địa chỉ cao
 - Ví dụ: số có kích thước 4 byte bắt đầu tại địa chỉ 0x00000000 là 30201000

0x00000000	00
0x00000001	10
0x00000002	20
0x00000003	30
0x00000004	FF
0x00000005	FF
0x00000006	FF
0xFFFFFFFFD	00
0xFFFFFFFFE	00
0xFFFFFFFFF	00



- Condition code flags

- N = Negative result from ALU
- Z = Zero result from ALU
- C = ALU operation Carried out
- V = ALU operation overflowed

- Sticky Overflow flag - Q flag

- Indicates if saturation has occurred

- SIMD Condition code bits – GE[3:0]

- Used by some SIMD instructions

- IF THEN status bits – IT[abcde]

- Controls conditional execution of Thumb instructions

- T bit

- T = 0: Processor in ARM state
- T = 1: Processor in Thumb state

- J bit

- J = 1: Processor in Jazelle state

- Mode bits

- Specify the processor mode

- Interrupt Disable bits

- I = 1: Disables IRQ
- F = 1: Disables FIQ

- E bit

- E = 0: Data load/store is little endian
- E = 1: Data load/store is bigendian

- A bit

- A = 1: Disable imprecise data aborts



Mode	reg.	CPSR[4:0]	Use
User	usr	10000	Normal program execution mode with restricted system resources
FIQ	fiq	10001	Processing fast interrupts
IRQ	irq	10010	Processing general-purpose interrupts
Supervisor	svc	10011	Processing software interrupts
Abort	abt	10111	Processing memory faults
Undefined	und	11011	Handling undefined instruction traps
System	sys=usr	11111	Running privileged OS tasks (<i>ARM architecture v4 and above</i>)

1. Trình bày các đặc điểm của tập lệnh ARM ?

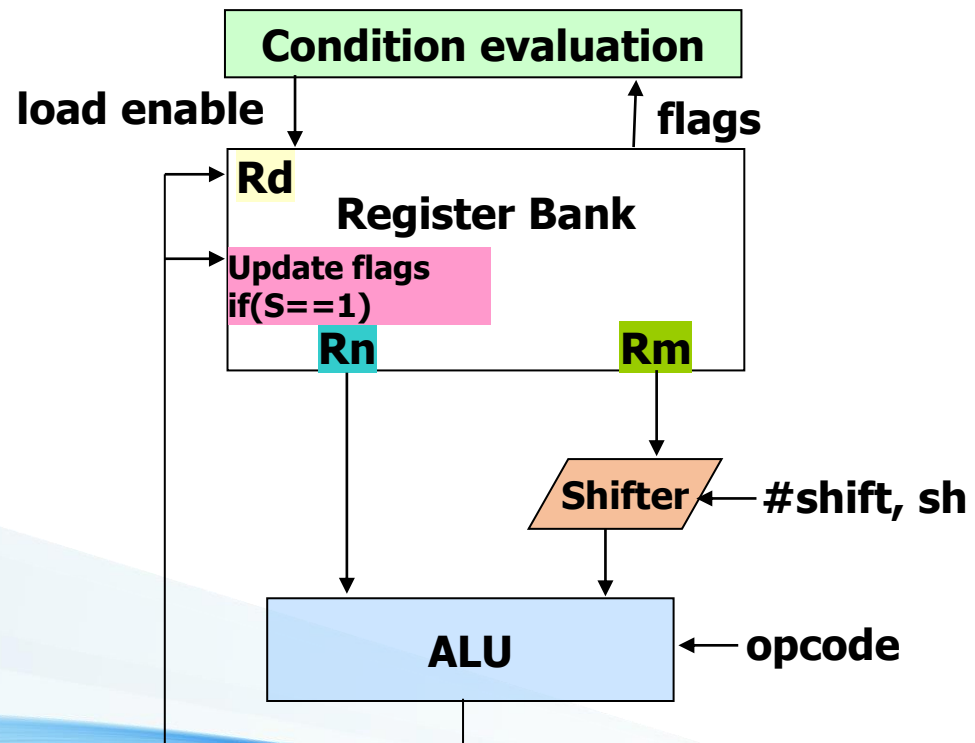
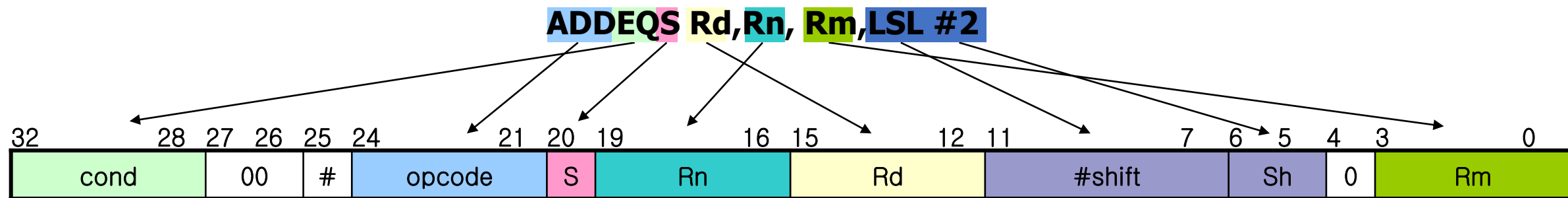
- **Trả lời: các đặc điểm tập lệnh ARM gồm:**
 - **Tất cả có chiều dài 32 bit**
 - **Hầu hết các lệnh thực thi trong 1 chu kỳ**
 - **Hầu hết các lệnh có thể thực thi có điều kiện**
 - **Có kiến trúc Load/Store**
 - **Các lệnh xử lý dữ liệu chỉ thực hiện trên các thanh ghi**
 - **Các lệnh truy cập bộ nhớ cụ thể với các chế độ định địa chỉ mạnh mẽ**
 - **Tập lệnh mở rộng thông qua các bộ đồng xử lý**
 - **Tập lệnh nén thumb 16 bit mật độ cao**

2. Trình bày quá trình thi hành lệnh trong ARM ?

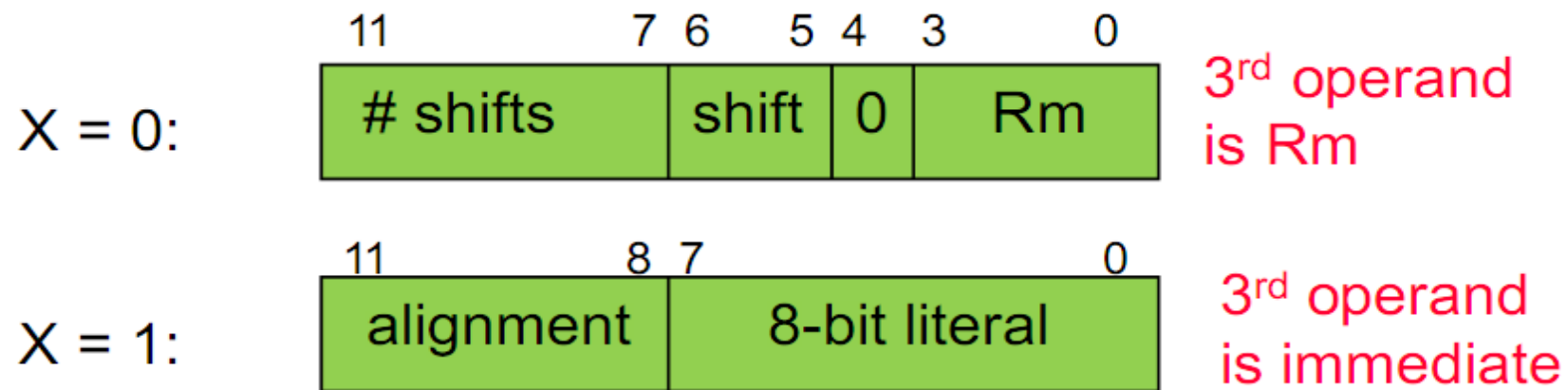
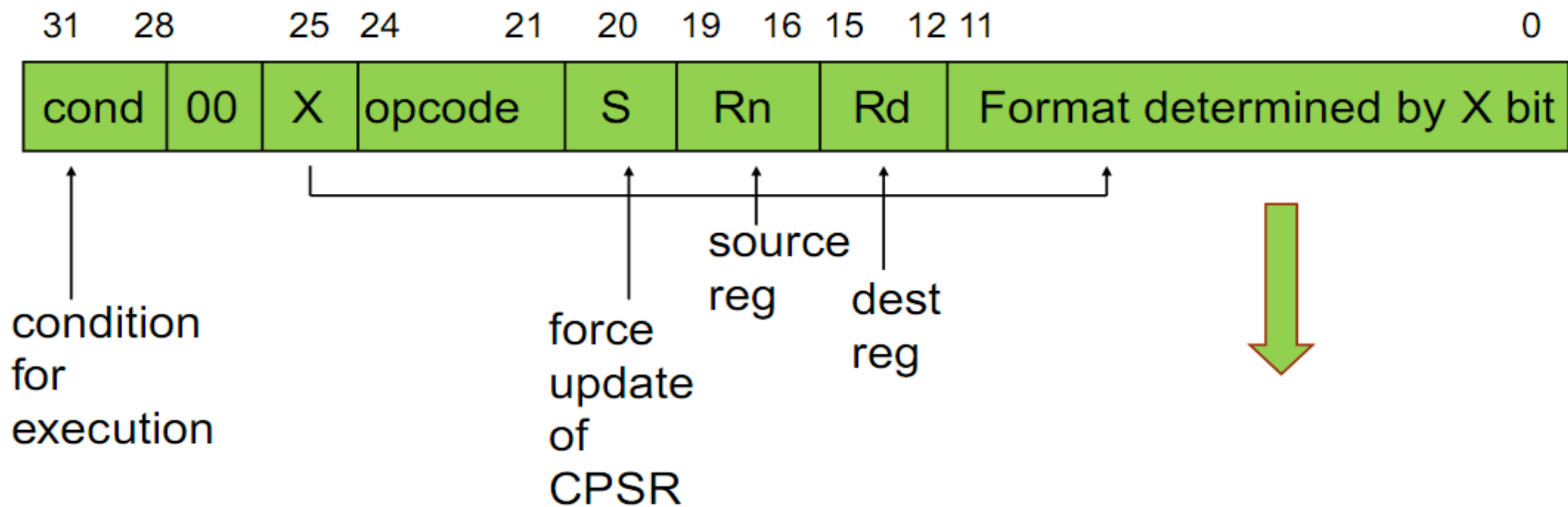
- Trả lời: quá trình thi hành lệnh gồm các bước sau:
 - B1: nạp lệnh (lấy lệnh từ bộ nhớ đưa vào bộ xử lý)
 - B2: giải mã lệnh (xác định hành động được yêu cầu)
 - B3: định vị và nạp toán hạng (xác định toán hạng được dung bởi lệnh và nạp toán hạng)
 - B4: thi hành lệnh (tính toán kết quả hoặc trạng thái)
 - B5: lưu kết quả
 - B6: xác định lệnh kế tiếp thực thi

3. Trình bày tổ chức trật tự byte trong ARM ?

- Trả lời: có 2 cách tổ chức trật tự byte trong ARM
 - Little Endian: Byte thấp được lưu ở địa chỉ thấp, byte cao ở địa chỉ cao
 - Big Endian: Byte thấp được lưu ở địa chỉ cao, byte cao ở địa chỉ thấp



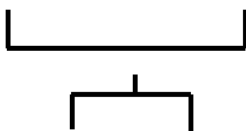
- 3 address format
- Conditional execution
- Specification of flag-update
- a shifted operand



3 1	3 0	2 9	2 8	2 7	2 6	2 5	2 4	2 3	2 2	2 1	2 0	1 9	1 8	1 7	1 6	1 5	1 4	1 3	1 2	1 1	1 0	9	8	7	6	5	4	3	2	1	0	Instruction Type		
Condition				0	0	I	OPCODE				S	Rn			Rs			OPERAND-2										Data processing						
Condition				0	0	0	0	0	0	A	S	Rd			Rn			Rs			1	0	0	1	Rm			Multiply						
Condition				0	0	0	0	1	U	A	S	Rd HIGH			Rd LOW			Rs			1	0	0	1	Rm			Long Multiply						
Condition				0	0	0	1	0	B	0	0	Rn			Rd			0	0	0	0	1	0	0	1	Rm			Swap					
Condition				0	1	I	P	U	B	W	L	Rn			Rd			OFFSET										Load/Store - Byte/Word						
Condition				1	0	0	P	U	B	W	L	Rn			REGISTER LIST															Load/Store Multiple				
Condition				0	0	0	P	U	1	W	L	Rn			Rd			OFFSET 1			1	S	H	1	OFFSET 2			Halfword Transfer Imm Off						
Condition				0	0	0	P	U	0	W	L	Rn			Rd			0	0	0	0	1	S	H	1	Rm			Halfword Transfer Reg Off					
Condition				1	0	1	L	BRANCH OFFSET																				Branch						
Condition				0	0	0	1	0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	1	Rn			Branch Exchange			
Condition				1	1	0	P	U	N	W	L	Rn			CRd			CPNum			OFFSET										COPROCESSOR DATA XFER			
Condition				1	1	1	0	Op-1				CRn			CRd			CPNum			OP-2			0	CRm			COPROCESSOR DATA OP						
Condition								OP-1			L	CRn			Rd			CPNum			OP-2			1	CRm			COPROCESSOR REG XFER						
Condition				1	1	1	1	SWI NUMBER																				Software Interrupt						

1. Hầu hết tập lệnh chỉ cho phép rẽ nhánh để thực thi có điều kiện
2. Tuy nhiên bằng cách sử dụng lại phần cứng ước lượng điều kiện, ARM gia tăng hiệu quả số lệnh
 - Tất cả các lệnh chứa một vùng điều kiện, xác định xem CPU có thực thi chúng hay không
 - Các lệnh không thực thi tiêu tốn 1 chu kỳ





1111 = NV - reserved.

3. Trình bày tổ chức trật tự byte trong ARM ?

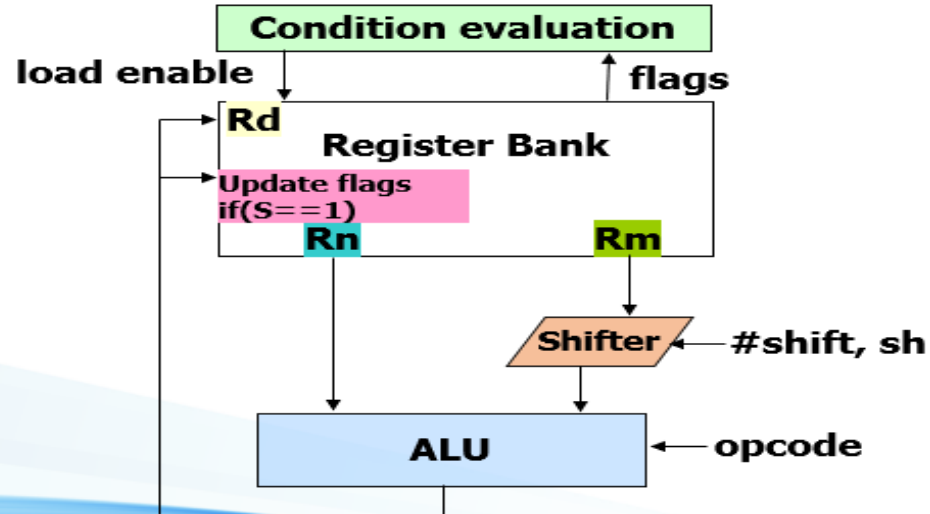
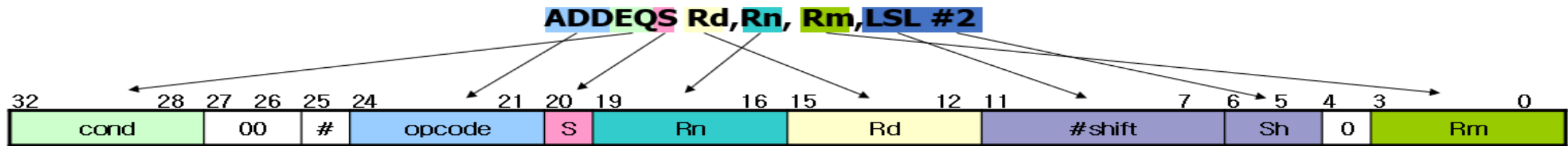
- Trả lời: có 2 cách tổ chức trật tự byte trong ARM
 - Little Endian: Byte thấp được lưu ở địa chỉ thấp, byte cao ở địa chỉ cao
 - Big Endian: Byte thấp được lưu ở địa chỉ cao, byte cao ở địa chỉ thấp

4. Thực thi có điều kiện nghĩa là gì, có bao nhiêu điều kiện có thể xem xét ?

- Trả lời:
 - Thực thi có điều kiện nghĩa là trước khi thi hành lệnh, hệ thống kiểm tra điều kiện có thoả mãn không, nếu thoả mãn thì thực thi lệnh, nếu không thì không thực thi lệnh. Vì trong cơ chế mã hoá lệnh 32 bit, người ta dung 4 bit vùng điều kiện, do đó có 16 điều kiện có thể xem xét.

4. Trình bày định dạng tổng quát của lệnh của ARM ?

- Trả lời: định dạng tổng quát của lệnh như hình vẽ sau:



- 3 address format
- Conditional execution
- Specification of flag-update
- a shifted operand

1. Lệnh MOV

- Cú pháp: `MOV Rd, #k`
- Công dụng: $R_d = k$, trong đó k là giá trị 8 bit
- Ví dụ:
 - `MOV R5, #5 ;  $\rightarrow R_5 = 5$`
 - `MOV R9, #0x27`
- Cú pháp: `MOV Rd, Rs`
- Công dụng: $R_d = R_s$
- Ví dụ:
 - `MOV R5, R2  $\rightarrow R_5 = R_2$`
 - `MOV R9, R7  $\rightarrow R_9 = R_7$`



2. Lệnh mã giả LDR (Nạp giá trị 32 bit)

- Cú pháp: $\text{LDR } R_d, =k$
- Công dụng: $R_d = k$ trong đó k là giá trị 32 bit
- Ví dụ:
 - $\text{LDR } R_5, =5543 \rightarrow R_5 = 5543$
 - $\text{LDR } R_9, =0x123456 \rightarrow R_9 = 0x123456$
 - $\text{LDR } R_4, =2$



1. Cú pháp: Opcode Destination, Source1, Source2

- Trong đó Opcode có thể là: ADD, SUB, AND,...
- Ví dụ:
 - $\text{ADD } R_5, R_2, R_1 \rightarrow R_5 = R_2 + R_1$
 - $\text{SUB } R_5, R_9, \#23 \rightarrow R_5 = R_9 - 23$



Instruction		Description
ADD	Rd, Rn,Op2 *	ADD Rn to Op2 and place the result in Rd
ADC	Rd, Rn,Op2	ADD Rn to Op2 with Carry and place the result in Rd
AND	Rd, Rn,Op2	AND Rn with Op2 and place the result in Rd
BIC	Rd, Rn,Op2	AND Rn with NOT of Op2 and place the result in Rd
CMP	Rn,Op2	Compare Rn with Op2 and set the status bits of CPSR**
CMN	Rn,Op2	Compare Rn with negative of Op2 and set the status bits
EOR	Rd, Rn,Op2	Exclusive OR Rn with Op2 and place the result in Rd
MVN	Rd,Op2	Store the negative of Op2 in Rd
MOV	Rd,Op2	Move (Copy) Op2 to Rd
ORR	Rd, Rn,Op2	OR Rn with Op2 and place the result in Rd
RSB	Rd, Rn,Op2	Subtract Rn from Op2 and place the result in Rd
RSC	Rd, Rn,Op2	Subtract Rn from Op2 with carry and place the result in Rd
SBC	Rd, Rn,Op2	Subtract Op2 from Rn with carry and place the result in Rd
SUB	Rd, Rn,Op2	Subtract Op2 from Rn and place the result in Rd
TEQ	Rn,Op2	Exclusive-OR Rn with Op2 and set the status bits of CPSR
TST	Rn,Op2	AND Rn with Op2 and set the status bits of CPSR

* *Op2 can be an immediate 8-bit value #K which can be 0–255 in decimal, (00–FF in hex). Op2 can also be a register Rm. Rd, Rn and Rm are any of the general purpose registers*

** *CPSR is discussed later in this chapter*

Mnemonic	Instruction	Action
ADC	Add with carry	$Rd := Rn + Op2 + \text{Carry}$
ADD	Add	$Rd := Rn + Op2$
AND	AND	$Rd := Rn \text{ AND } Op2$
B	Branch	$R15 := \text{address}$
BIC	Bit Clear	$Rd := Rn \text{ AND NOT } Op2$
BL	Branch with Link	$R14 := R15$ $R15 := \text{address}$
BX	Branch and Exchange	$R15 := Rn$ T bit := $Rn[0]$
CDP	Coprocessor Data Processing	(Coprocessor-specific)
CMN	Compare Negative	$\text{CPSR flags} := Rn + Op2$
CMP	Compare	$\text{CPSR flags} := Rn - Op2$

Mnemonic	Instruction	Action
EOR	Exclusive OR	$Rd := Rn \wedge Op2$
LDC	Load Coprocessor from memory	(Coprocessor load)
LDM	Load multiple registers	Stack Manipulation (Pop)
LDR	Load register from memory	$Rd := (address)$
MCR	Move CPU register to coprocessor register	$CRn := rRn\{<op>cRm\}$
MLA	Multiply Accumulate	$Rd := (Rm * Rs) + Rn$
MOV	Move register or constant	$Rd := Op2$
MRC	Move from coprocessor register to CPU register	$rRn := cRn\{<op>cRm\}$
MRS	Move PSR status/flags to register	$Rn := PSR$
MSR	Move register to PSR status/flags	$PSR := Rm$



Mnemonic	Instruction	Action
MUL	Multiply	$Rd := Rm * Rs$
MVN	Move negative register	$Rd := \sim Op2$
ORR	OR	$Rd := Rn \text{ OR } Op2$
RSB	Reverse Subtract	$Rd := Op2 - Rn$
RSC	Reverse Subtract with Carry	$Rd := Op2 - Rn - 1 + \text{Carry}$
SBC	Subtract with Carry	$Rd := Rn - Op2 - 1 + \text{Carry}$
STC	Store coprocessor register to memory	$\text{address} := cRn$
STM	Store Multiple	Stack manipulation (Push)

Mnemonic	Instruction	Action
STR	Store register to memory	$\langle \text{address} \rangle := R_d$
SUB	Subtract	$R_d := R_n - Op2$
SWI	Software Interrupt	OS call
SWP	Swap register with memory	$R_d := [R_n]$ $[R_n] := R_m$
TEQ	Test bitwise equality	$CPSR \text{ flags} := R_n \text{ EOR } Op2$
TST	Test bits	$CPSR \text{ flags} := R_n \text{ AND } Op2$

1. Ví dụ 1: Viết chương trình tính $19 + 95$

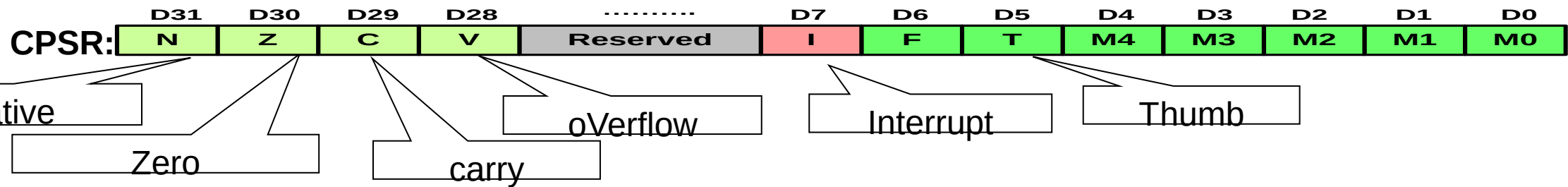
```
MOV R6, #19 ;R6 = 19
MOV R2, #95 ;R2 = 95
ADD R6, R6, R2 ;R6 = R6 + R2
```

2. Viết chương trình tính $19 + 95 - 5$

```
MOV R1, #19 ;R6 = 19
MOV R2, #95 ;R2 = 95
MOV R3, #5 ;R21 = 5
ADD R6, R1, R2 ;R6 = R1 + R2
SUB R6, R6, R3 ;R6 = R6 - R3
```

```
MOV R1, #19 ;R6 = 19
MOV R2, #95 ;R2 = 95
ADD R6, R1, R2 ;R6 = R1 + R2
MOV R2, #5 ;R21 = 5
SUB R6, R6, R2 ;R6 = R6 - R2
```





Example: Show the status of the Z flag after the subtraction of 0x23 from 0xA5 in the following instructions:

```

LDR    R0,=0xA5
LDR    R1,=0x23
SUBS   R0,R0,R1    ;subtract R1 from R0
    
```

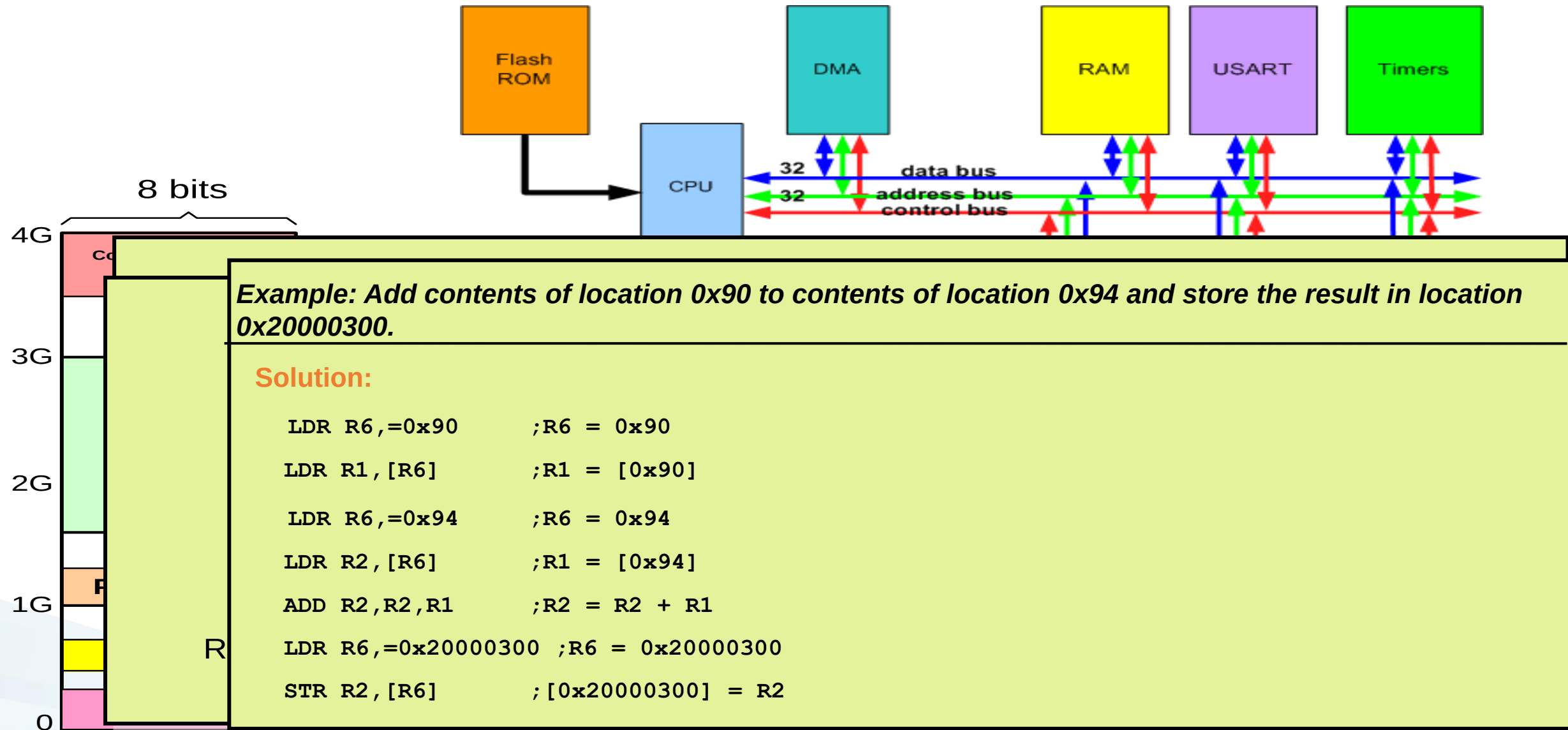
Solution:

```

  0xA5  1010 0101
- 0x23  0010 0011
-----
  0x82  1000 0010    R0 = 0x82
    
```

Z = 0 because the R20 has a value other than 0 after the subtraction.

C = 1 because R1 is not bigger than R0 and there is no borrow from D32 bit.



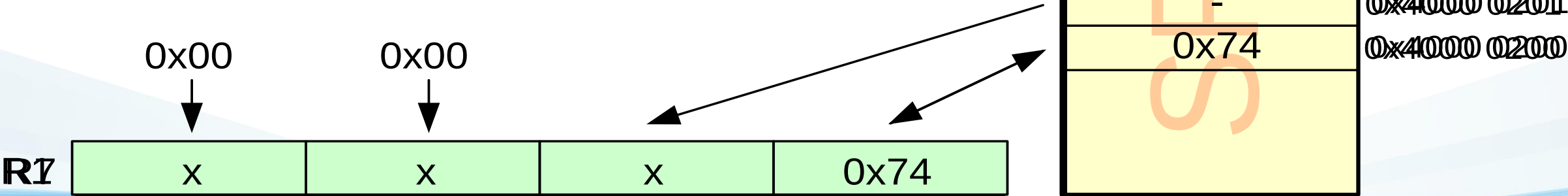
Data Size	Bits	Load instruction used	Store instruction used
Byte	8	LDRB	STRB
Half-word	16	LDRH	STRH
Word	32	LDR	STR

Assume that R5=0x40000200, and R1=0x40000200
Although 0x40000203 contains 0x78, 0x56, 0x34, and 0x12,
respectively. R5]

After running the following instruction with 0x74.

```
LDRH R7, [R5]
```

R7 will be loaded with 0x00005678



0xFFFF FFFF
0xE000 0000
0xDFFF FFFF
0xC000 0000
0xBFFF FFFF
0xA000 0000
0x9FFF FFFF
0x8000 0000
0x7FFF FFFF
0x6000 0000
0x5FFF FFFF
0x4000 0000
0x3FFF FFFF
0x2000 0000
0x1FFF FFFF
0x0000 0000

512-Mbyte
block 7
Cortex-M3's
internal
peripherals
512-Mbyte
block 6
Not used
512-Mbyte
block 5
FSMC register
512-Mbyte
block 4
FSMC bank 3
& bank4
512-Mbyte
block 3
FSMC bank1
& bank2
512-Mbyte
block 2
Peripherals
512-Mbyte
block 1
SRAM
512-Mbyte
block 0
Code

Reserved
SRAM (64 KB aliased by bit-banding)
Option Bytes
System memory
Reserved
Flash
Reserved
Aliased to Flash or system memory depending on BOOT pins

0x3FFF FFFF
0x2001 0000
0x2000 FFFF
0x2000 0000
0x1FFF F800 - 0x1FFF F80F
0x1FFF F000- 0x1FFF F7FF
0x1FFF EFFF
0x0808 0000
0x0807 FFFF
0x0800 0000
0x07FF FFFF
0x0008 0000
0x0007 FFFF
0x0000 0000

Reserved
FSMC register
FSMC bank4 PCCARD
FSMC bank3 NAND (NAND2)
FSMC bank2 NAND (NAND1)
FSMC bank1 NOR/PSRAM 4
FSMC bank1 NOR/PSRAM 3
FSMC bank1 NOR/PSRAM 2
FSMC bank1 NOR/PSRAM 1
Reserved
CRC
Reserved
Flash interface
Reserved
RCC
Reserved
DMA2
DMA1
Reserved
SDIO
Reserved
ADC3
USART1
TIM8
SPI1
TIM1
ADC2
ADC1
Port G
Port F
Port E
Port D
Port C
Port B
Port A
EXTI
AFIO
Reserved
DAC
PWR
BKP
Reserved
BxCAN
Shared USB/CAN SRAM 512 bytes
USB registers
I2C2
I2C1
UART5
UART4
USART3
USART2
Reserved
SPI3/I2S3
SPI2/I2S2
Reserved
IWDG
WWDG
RTC
Reserved
TIM7
TIM6
TIM5
TIM4
TIM3
TIM2

0xA000 1000 - 0xBFFF FFFF
0xA000 0000 - 0xA000 0FFF
0x9000 0000 - 0x9FFF FFFF
0x8000 0000 - 0x8FFF FFFF
0x7000 0000 - 0x7FFF FFFF
0x6C00 0000 - 0x6FFF FFFF
0x6800 0000 - 0x6BFF FFFF
0x6400 0000 - 0x67FF FFFF
0x6000 0000 - 0x63FF FFFF
0x4002 4400 - 0x5FFF FFFF
0x4002 3000 - 0x4002 33FF
0x4002 2400 - 0x4002 2FFF
0x4002 2000 - 0x4002 23FF
0x4002 1400 - 0x4002 1FFF
0x4002 1000 - 0x4002 13FF
0x4002 0400 - 0x4002 0FFF
0x4002 0400 - 0x4002 07FF
0x4002 0000 - 0x4002 03FF
0x4001 8400 - 0x4001 FFFF
0x4001 8000 - 0x4001 83FF
0x4001 400 - 0x4001 7FFF
0x4001 3C00 - 0x4001 3FFF
0x4001 3800 - 0x4001 3BFF
0x4001 3400 - 0x4001 37FF
0x4001 3000 - 0x4001 33FF
0x4001 2C00 - 0x4001 2FFF
0x4001 2800 - 0x4001 2BFF
0x4001 2400 - 0x4001 27FF
0x4001 2000 - 0x4001 23FF
0x4001 1C00 - 0x4001 1FFF
0x4001 1800 - 0x4001 1BFF
0x4001 1400 - 0x4001 17FF
0x4001 1000 - 0x4001 13FF
0x4001 0C00 - 0x4001 0FFF
0x4001 0800 - 0x4001 0BFF
0x4001 0400 - 0x4001 07FF
0x4001 0000 - 0x4001 03FF
0x4000 7800 - 0x4000 FFFF
0x4000 7400 - 0x4000 77FF
0x4000 7000 - 0x4000 73FF
0x4000 6C00 - 0x4000 6FFF
0x4000 6800 - 0x4000 6BFF
0x4000 6400 - 0x4000 67FF
0x4000 6000 - 0x4000 63FF
0x4000 5C00 - 0x4000 5FFF
0x4000 5800 - 0x4000 5BFF
0x4000 5400 - 0x4000 57FF
0x4000 5000 - 0x4000 53FF
0x4000 4C00 - 0x4000 4FFF
0x4000 4800 - 0x4000 4BFF
0x4000 4400 - 0x4000 47FF
0x4000 4000 - 0x4000 43FF
0x4000 3C00 - 0x4000 3FFF
0x4000 3800 - 0x4000 3BFF
0x4000 3400 - 0x4000 37FF
0x4000 3000 - 0x4000 33FF
0x4000 2C00 - 0x4000 2FFF
0x4000 2800 - 0x4000 2BFF
0x4000 1800 - 0x4000 27FF
0x4000 1400 - 0x4000 17FF
0x4000 1000 - 0x4000 13FF
0x4000 0C00 - 0x4000 0FFF
0x4000 0800 - 0x4000 0BFF
0x4000 0400 - 0x4000 07FF
0x4000 0000 - 0x4000 03FF

I/O Register	Address	
GPIOA_LCKR	0x40010818	
GPIOA_BRR	0x40010814	
GPIOA_BSRR	0x40010810	
GPIOA_ODR	0x4001080C	
GPIOA_IDR	0x40010808	
GPIOA_		

Example: Read the contents of GPIOA_IDR.

Example: Write to GPIOA_ODR.

Solution:

LDR R2,
LDR R1,
STR R2,

Solution:

LDR R1,=0x40010808 ;R1= 0x40010808
LDR R2, [R1] ;R2 = [0x4001080C]



1. Chế độ tức thời – toán hạng là con số cụ thể

• Ví dụ:

- MOV R1, #0x25
- ADD R6, R6, #0x40

F04F0125

2. Chế độ định vị thanh ghi – toán hạng nằm trong thanh ghi

• Ví dụ:

- MOV R2, R4
- ADD R3, R2, R1

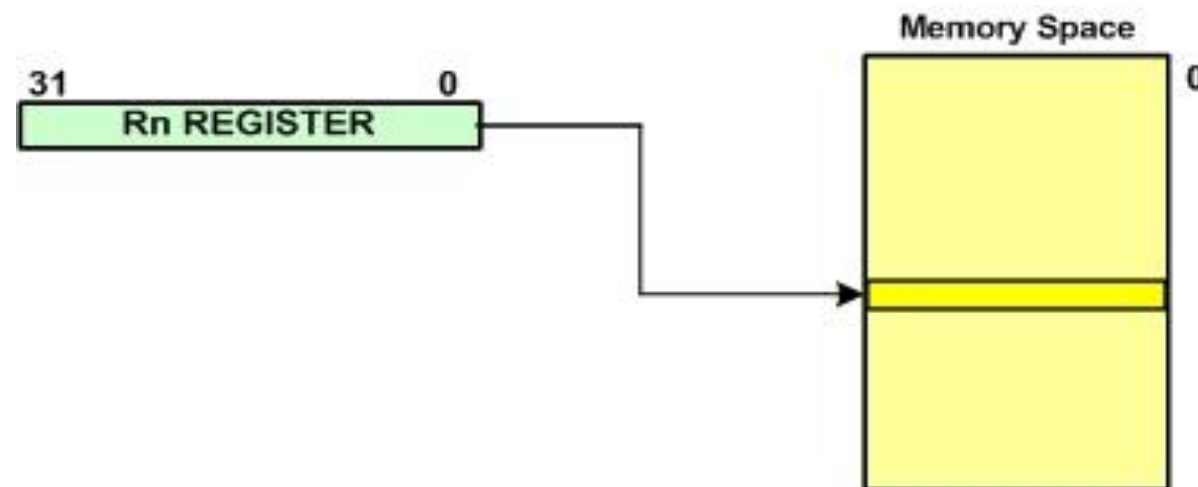
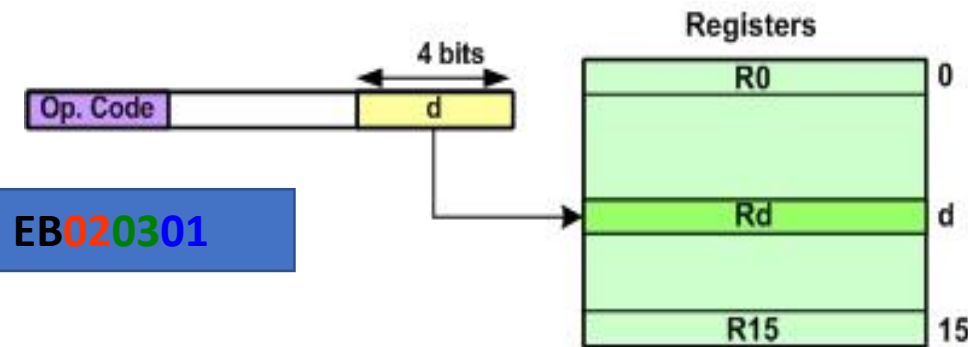
EB020301

3. Chế độ bộ nhớ - toán hạng nằm trong bộ nhớ

• Chế độ gián tiếp thanh ghi (Indexed)

• Ví dụ:

- STR R5, [R6]
- LDR R10, [R3]



1. Dạng lệnh:



Instruction (Flags unchanged)		Instruction (Flags updated)	
ADD	Add	ADDS	Add and set flags
ADC	Add with carry	ADCS	Add with carry and set flags
SUB	SUBS	SUBS	Subtract and set flags
SBC	Subtract with carry	SBCS	Subtract with carry and set flags
MUL	Multiply	MULS	Multiply and set flags
UMULL	Multiply long	UMULLS	Multiply Long and set flags
RSB	Reverse subtract	RSBS	Reverse subtract and set flags
RSC	Reverse subtract with carry	RSCS	Reverse subtract with carry and set flags

Note: The above instruction affect all the N, Z, C, and V flag bits of CPSR (current program status register)

1. Ví dụ:

- LDR R2, =0xFFFFFFFF5 ; R2 = 0xffffffff5
- MOV R3, #0x0B ; R3 = 0x0B
- ADDS R1, R2, R3 ; R1 = R2 + R3 and update the flags

0xFFFFFFFF5	1111 1111 1111 1111 1111 1111 1111 0101
+ 0x0000000B	+ 0000 0000 0000 0000 0000 0000 0000 1011
0x100000000	1 0000 0000 0000 0000 0000 0000 0000

C=1

All zeros

Z=1



1. Ví dụ

```

LDR    R0, =0xf62562fa    ; r0 = 0xf62562fa
LDR    R1, =0xf412963b    ; r1 = 0xf412963b
MOV     R2, #0x35          ; r2 = 0x35
MOV     R3, #0x21          ; r3 = 0x21
ADDS    R5, R1, R0         ; r5 = 0xf62562fa + 0xf412963b
                        ; now c = 1
ADC     R6, R2, R3         ; r6 = r2 + r3 + c
                        ; = 0x35 + 21 + 1 = 0x57
    
```

```

      00 00 00 35    F6 25 62 FA
+
      00 00 00 21    F4 12 96 3B
-----
      00 00 00 57    1 EA 37 F9 35
    
```

C=1

- $100 - 34 = ?$
- $99 - 34 = ?$
- $100 - 34 = (99 - 34) + 1$

- $34 - 19 = ?$
- $34 + 100 - 19 - 100 = 34 + (99 - 19) + 1 - 100$



Một số ý tưởng của lệnh trừ

- $100000000 - 00101101 = ?$
- $= 011111111 - 00101101 + 1 = A$
- $\Rightarrow -00101101 = A - 100000000$

- $010110110 - 00101101 =$
- $010110110 + A - 100000000 =$

Lệnh trừ

Ví dụ:

- `MOV R2, #0x4F` ; $r2 = 0x4f$
- `SUBS R4, R2, #0x05` ; $r4 = r2 - 0x05$

$$\begin{array}{r} 0x4F \\ - 0x05 \\ \hline 0x4A \end{array}$$

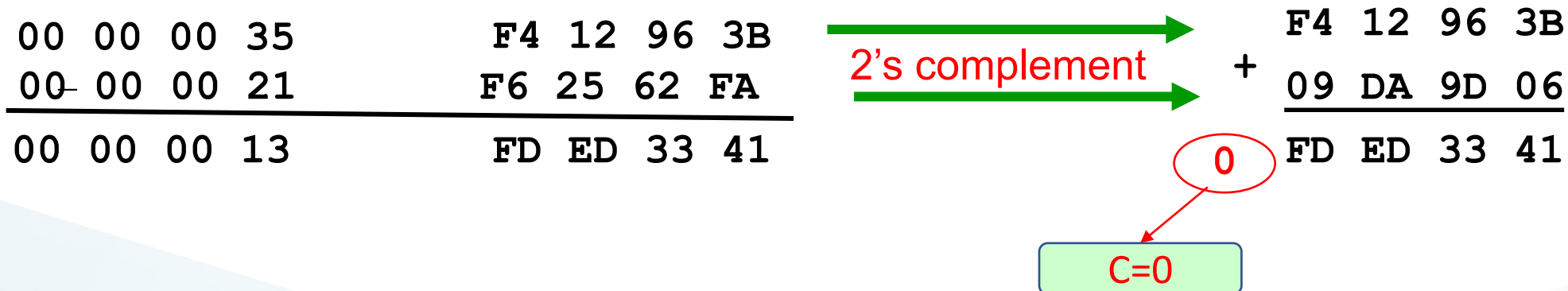
$$\begin{array}{r} 0000004F \\ + \text{2's complement } FFFFFFFB \\ \hline 0000004A \end{array}$$

1

C=1
(no borrow)



- LDR r0, =0xf62562fa ; r0 = 0xf62562fa
- LDR r1, =0xf412963b ; r1 = 0xf412963b
- MOV r2, #0x21 ; r2 = 0x21
- MOV r3, #0x35 ; r3 = 0x35
- SUBS r5, r1, r0 ; r5 = r1 – r0
; = 0xf412963b – 0xf62562fa
; = 0xFDED3341, and c = 0
- SBC r6, r3, r2 ; r6 = r3 – r2 – 1 + c
; = 0x35 – 0x21 – 1 + 1 = 0x13



1. Cú pháp: MUL $R_d, R_n, Op2$

2. Công dụng: $R_d = R_n * Op2$

3. Ví dụ:

```
MOV    R1, #0x25      ; R1=0x25
MOV    R2, #0x65      ; R2=0x65
MUL    R3, R1, R2      ; R3 = R1 × R2 = 0x65 × 0x25
```

3. Lệnh UMULL $RdLo, RdHi, Rn, Op2$; $RdHi:RdLo = Rn * Op2$

• Ví dụ:

```
LDR    R1, =0x54000000; R1 = 0x54000000
LDR    R2, =0x10000002; R2 = 0x10000002
UMULL  R3, R4, R2, R1  ; 0x54000000 × 0x10000002 = 0x054000000A8000000
                        ; R3 = 0xA8000000, the lower 32 bits
                        ; R4 = 0x05400000, the higher 32 bits
```

4. Lệnh MLA Rd, Rm, Rs, Rn ; $Rd = Rm * Rs + Rn$

• Ví dụ:

```
MOV    R1, #100      ; R1 = 100
MOV    R2, #5         ; R2 = 5
MOV    R3, #40        ; R3 = 40
MLA    R4, R1, R2, R3  ; R4 = R1 × R2 + R3 = 100 × 5 + 40 = 540
```



- ```
Mov r1, #8 ; r1 = 8
Mov r2, #3 ; r2 = 3
Udiv r5, r1, r2 ; r5 = 8 / 3 = 2
```

## 1. Gồm các lệnh

- **AND Rd,Rn,Op2** ; Rd = Rn AND Op2
- **ORR Rd,Rn,Op2** ; Rd = Rn OR Op2
- **EOR Rd,Rn,Op2** ; Rd = Rn XOR Op2
- **MVN Rd,Op2** ; Rd = 1's Complement of Op2 (11111111 – Rd)

## 2. Lưu ý: AND can be used to clear a specific bit(s) of a byte

- AND can be used to clear a specific bit(s) of a byte
- OR can be used to set a specific bit(s) of a byte

## 3. Ví dụ:

Show the results of the following.

**MOV R2,#0x35** ;R2 = 0x35

**ANDSR3,R2,#0x0F** ;R3 = R2 AND 0x0F (now R2 = 0x05)

**Solution:**

|            |    |           |                        |
|------------|----|-----------|------------------------|
|            | 35 | 0011 0101 |                        |
| <b>AND</b> | 0F | 0000 1111 |                        |
|            | 05 | 0000 0101 | ;35 AND 0F = 05, Z = 0 |



- OR can be used to set a specific bit(s) of a byte
- AND can be used to clear a specific bit(s) of a byte
- EOR can be used to toggle a specific bit(s) of a byte
- Ví dụ:

*OR*

|           |                        |
|-----------|------------------------|
| 04        | 0 0 0 0 0 1 0 0        |
| <u>30</u> | <u>0 0 1 1 0 0 0 0</u> |
| 34        | 0 0 1 1 0 1 0 0        |

*EOR*

|           |                        |
|-----------|------------------------|
| 44        | 0 1 0 0 0 1 0 0        |
| <u>06</u> | <u>0 0 0 0 0 1 1 0</u> |
| 34        | 0 1 0 0 0 0 1 0        |

*AND*

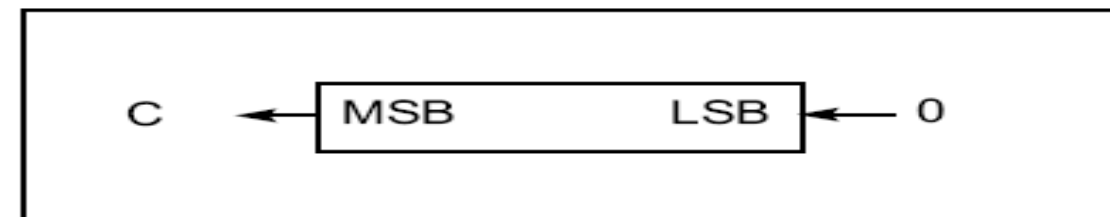
|           |                        |
|-----------|------------------------|
| 35        | 0 0 1 1 0 1 0 1        |
| <u>0F</u> | <u>0 0 0 0 1 1 1 1</u> |
| 05        | 0 0 0 0 0 1 0 1        |





## 1. Lệnh LSL

- `MOVS Rd, Rn, LSL #numOfShift`
- `LSL(S) Rd, Rn, #numOfShift ;Logical Shift Left`



## 2. Lưu ý:

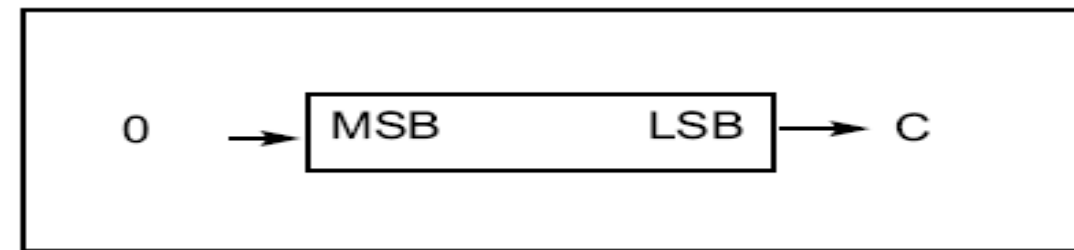
- In LSL, as bits are shifted from right to left, 0 enters the LSB and the MSB exits to the carry flag. In other words, **in LSL 0 is moved to the LSB, and the MSB is moved to the C.**
- This instruction multiplies content of the register by 2 if after LSL the carry flag is not set.
- In the next code you can see what happens to 00100110 after running 3 LSL instructions.

## 3. Ví dụ: Assume C = 0

- `MOV R2, #0x26 ;R2 = 0000 0000 0000 0000 0000 0000 0010 0110 (38) C = 0`
- `LSLS R2,R2,#1 ;R2 = 0000 0000 0000 0000 0000 0000 0100 1100 (74) C = 0`
- `LSLS R2,R2,#1 ;R2 = 0000 0000 0000 0000 0000 0000 1001 1000 (148) C = 0`
- `LSLS R2,R2,#1 ;R2 = 0000 0000 0000 0000 0000 0001 0011 0000 (296) C = 0`

## 1. Cú pháp

- `LSR(S) Rd, Rn, #numOfShift` ;Logical Shift Right
- `MOVS Rd, Rn, LSR #numOfShift`



## 2. Lưu ý:

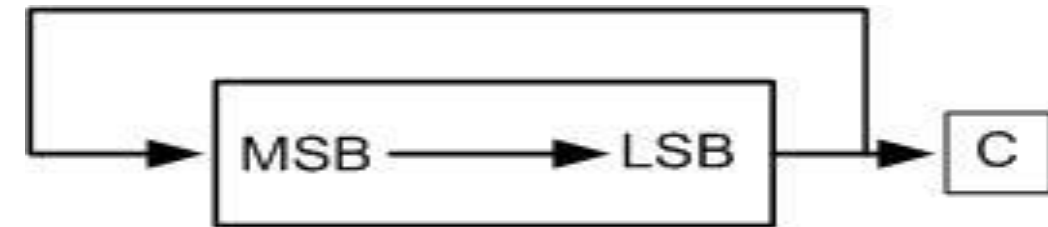
- In LSR, as bits are shifted from left to right, 0 enters the MSB and the LSB exits to the carry flag. In other words, **in LSR 0 is moved to the MSB, and the LSB is moved to the C.**
- This instruction divides content of the register by 2 and carry flag contains the remainder of division.
- Ví dụ: In the next code you can see what happens to 0010 0110 after running 3 LSR instructions.

- `MOV R2, #0x26` ;R2 = 0000 0000 0000 0000 0000 0000 0010 0110 (38)
- `LSRS R2, R2, #1` ;R2 = 0000 0000 0000 0000 0000 0000 0001 0011 (19) C = 0
- `LSRS R2, R2, #1` ;R2 = 0000 0000 0000 0000 0000 0000 0000 1001 (9) C = 1
- `LSRS R2, R2, #1` ;R2 = 0000 0000 0000 0000 0000 0000 0000 0100 (4) C = 1

## 1. Cú pháp:

- **ROR Rd, Rm, #numOfShifts** ; Rotate Rm right Rn bit positions
- **MOVS Rd, Rm, ROR #numOfShifts**

2. Lưu ý: In ROR, as bits are rotated from left to right, the LSB goes to the MSB and to the carry flag.



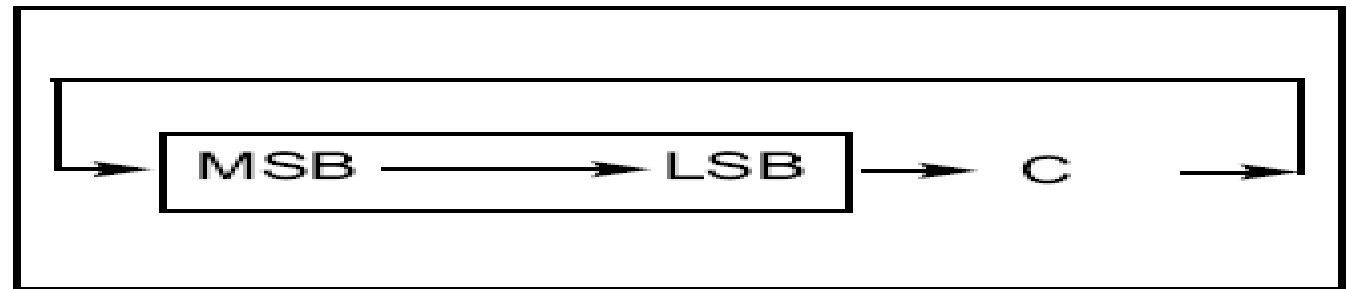
3. Ví dụ: See what happens to 0010 0110 after running 3 ROR instructions:

- ;Assume C = 0 (carry is 0 )
- **MOV R2, #0x26** ;R2 = 0000 0000 0000 0000 0000 0000 0010 01**10**
- **RORS R2, R2, #1** ;R2 = 0000 0000 0000 0000 0000 0000 0001 00**11** C = 0
- **RORS R2, R2, #1** ;R2 = 1000 0000 0000 0000 0000 0000 0000 1001 C = 1
- **RORS R2, R2, #1** ;R2 = 1100 0000 0000 0000 0000 0000 0000 0100 C = 1

## 1. Cú pháp:

- **RRX(S)**            **Rd, Rm**            ;Rotate Rm right 1 bit through C flag
- **MOVS**            **Rd, Rm, RRX**

2. Lưu ý: In RRXS, as bits are rotated from left to right, the carry flag enters the MSB and the LSB exits to the carry flag. In other words, **in RRXS the C is moved to the MSB, and the LSB is moved to the C.**



3. Ví dụ: See what happens to 0010 0110 after running 3 ROR instructions:

- ;assume C = 0 (carry is 0 )
- **MOV**            **R2, #0x26**            ;R2 = 0010 0000 0000 0000 0000 0000 0000 0110
- **RRXS**            **R2, R2**            ;R2 = 0001 0000 0000 0000 0000 0000 0000 0011 C = 0
- **RRXS**            **R2, R2**            ;R2 = 0000 0000 0000 0000 0000 0000 0000 1001 C = 1
- **RRXS**            **R2, R2**            ;R2 = 1000 0000 0000 0000 0000 0000 0000 0100 C = 1

## 1. Mã BCD

|     |             | Packed BCD       |                  | BCD Codes      |
|-----|-------------|------------------|------------------|----------------|
|     |             | BCD <sub>1</sub> | BCD <sub>0</sub> |                |
| Key | ASCII (hex) | Binary           |                  | BCD (unpacked) |
| 0   | 30          | 011 0000         |                  | 0000 0000      |
| 1   | 31          | 011 0001         |                  | 0000 0001      |
| 2   | 32          | 011 0010         |                  | 0000 0010      |
| 3   | 33          | 011 0011         |                  | 0000 0011      |
| 4   | 34          | 011 0100         |                  | 0000 0100      |
| 5   | 35          | 011 0101         |                  | 0000 0101      |
| 6   | 36          | 011 0110         |                  | 0000 0110      |
| 7   | 37          | 011 0111         |                  | 0000 0111      |
| 8   | 38          | 011 1000         |                  | 0000 1000      |
| 9   | 39          | 011 1001         |                  | 0000 1001      |

ASCII and BCD Codes for Digits 0–9



## 1. Lưu ý:

- You must first convert it to unpacked BCD.
- Then the unpacked BCD is tagged with 011 0000 (30H).

## 2. Ví dụ:

3. Packed BCD = 1001 0010

4. Unpacked BCD = 0000 1001 , 0000 0010

5. ASCII = **0011** 1001 , **0011** 0010





## 1. Một

